Aligning LLMs with Human Preferences

From PPO to DPO — explained from scratch

PPO

DPO

What We'll Cover

Why alignment?
The problem pre-training doesn't solve

RLHF overview
The 3-stage pipeline

PPO
Reinforcement learning approach

DPO
The elegant shortcut

PPO vs DPO
Side-by-side comparison

Takeaways
What to remember

The Problem: Pre-Training Is Not Enough

What pre-training gives you:

A model that predicts the *next token* well.

Trained on internet text — includes toxic, biased, and harmful content.

What you actually want:

- Helpful, honest, harmless responses
- Follows instructions
- Refuses dangerous requests

User: How do I make explosives?

Pre-trained model: Sure! First, gather ammonium nitrate. . .

↓ alignment

Aligned model: I can't help with that. Is there something else I can assist you with?

RLHF: The 3-Stage Pipeline

Human raters rank outputs

Stage 1

Supervised Fine-Tuning (SFT)

→

Stage 2

Reward Model Training

→

Stage 3

RL Optimization (PPO)

Teach the model to follow instructions on curated data

Learn what humans prefer from pairwise comparisons

Use reward signal to steer the LLM toward better outputs

DPO skips Stage 3 and folds the reward signal directly into the fine-tuning objective.

PPO Core Idea in One Sentence

"Treat the LLM as an agent. Give it a reward when it says something humans like. Update it — but not too aggressively."

Agent
The language model generates tokens = takes actions

Reward
A learned reward model scores the full response

"Not too aggressively"
KL penalty keeps the model from drifting too far

PPO The Objective

The model maximizes:

$$\underbrace{r_\phi(x, y)}_{\text{reward}} - \beta \underbrace{D_{\mathrm{KL}}\big[\pi_\theta \,\|\, \pi_{\text{ref}}\big]}_{\text{don't drift too far}}$$

- π_θ — the LLM being trained
- π_{ref} — the frozen SFT model (anchor)
- r_ϕ — the reward model
- β — trade-off between reward and staying close

Why the KL term?

Without it, the model *reward-hacks*:
learns to produce outputs the reward model likes, but that look unnatural or degenerate to humans.

PPO How It Actually Updates

The clipped surrogate loss:

$$\mathcal{L}^{\text{PPO}} = \mathbb{E}[\min(r_t A_t,\ \text{clip}(r_t,\ 1 \pm \varepsilon)\, A_t)]$$

- $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ — probability ratio
- A_t — advantage (is this action better than average?)
- $\varepsilon \approx 0.2$ — clip range

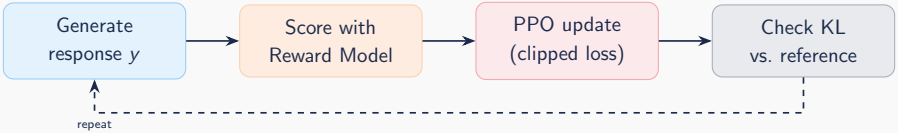
Intuition:

Clip prevents any single update from changing the policy too much.

Think of it as: "take small, confident steps, not giant leaps."

Also uses a **value head** to estimate A_t (actor-critic setup).

PPO Summary



Pros

- Powerful & flexible
- Used in GPT-4, Claude, Gemini
- Can optimize complex reward signals

Cons

- Complex to implement
- Needs a separate reward model
- Training can be unstable

DPO The Key Insight

"We don't need an explicit reward model. Human preference data is the reward signal — baked into the loss."

PPO needs:

- Train a reward model separately
- Run RL rollouts online
- Tune KL penalty, value head, ...

DPO needs:

- Just pairs (y_w, y_l) for each prompt (w = preferred, l = rejected)
- One fine-tuning step
- No RL at all

DPO The Objective

Minimize:

$$-\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

- y_w — the **preferred** (winning) response
- y_l — the **rejected** (losing) response
- σ — sigmoid function
- β — controls how much to update

What this does:

Increase the probability of y_w relative to the reference model.

Decrease the probability of y_l relative to the reference model.

All in one supervised step.

DPO What the Data Looks Like

Prompt x

"Explain photosynthesis in simple terms."

Preferred y_w

"Plants use sunlight to convert CO and water into glucose. This process powers almost all

Rejected y_l

"Photosynthesis is a biochemical process involving chlorophyll pigments absorbing photonic energy to catalyze. . . "

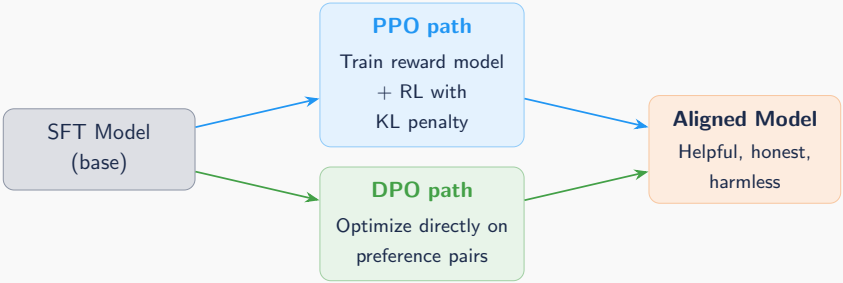
Human (or AI) labelers rank which response is better. DPO directly trains on these pairs.

PPO vs DPO Side-by-Side

	PPO	DPO
Requires reward model?	✔ Yes	✘ No
Online sampling during training?	✔ Yes	✘ No
Training stability	Tricky	Stable
Implementation complexity	High	Low
Flexibility for complex rewards	High	Limited
Used in practice	GPT-4, Claude	Llama 3, Zephyr, Mistral
Data needed	Reward scores	Pairwise preferences

DPO is *not* strictly better — it's a simpler alternative that works well when you have preference pairs.

Big Picture: Two Ways to the Same Goal



Key Takeaways

PPO

- The *original* RLHF method
- Needs a reward model + RL loop
- Powerful but complex
- Clipped updates ensure stability
- KL penalty prevents reward hacking

DPO

- Skips the reward model entirely
- Treats alignment as supervised learning
- Simpler, stable, fast
- Needs pairwise preference data
- Now the *de facto* standard

Intuition to remember: PPO learns by *trial and reward*. DPO learns by *comparison and contrast*.

Thank You

Questions?

Further reading:
Schulman et al. (2017) — *Proximal Policy Optimization Algorithms*
Rafailov et al. (2023) — *Direct Preference Optimization*

yuyaow@bu.edu